ada3cf64-feb7-4a12-ab99-cf5db4162c5f

<current>

<request>

You are working on an issue in the 'microsoft/regorus' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/regorus/regorus (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-06-23T15:01:38.765+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section details the Dependabot vulnerability alert you should resolve

<alert_title>PyO3 has a missing `Sync` bound on `PyCFunction::new_closure` closures</alert_title>
<alert_description>`PyCFunction::new_closure` (and the temporary `new_closure_bound` complement in the 0.21–0.22 series) required the supplied closure to be `Send + 'static` but not `Sync`. The resulting `PyCFunction` is a Python callable that can be invoked from any Python thread, which means the closure may be called concurrently from multiple threads, and needs a `Sync` bound to prevent possible data races.

The problem exists under all Python versions but is particularly vulnerable under the newer free-threaded Python variant, which do not have serial execution imposed by the Global Interpreter Lock. Under releases protected by the GIL, the ability to "detach" from the Python interpreter temporarily inside the closure (e.g. by `Python::detach`) makes it possible for interleaved and/or concurrent execution of various portions of

the closure.

PyO3 0.29.0 added a `Sync` bound to close this thread-safety bug.</alert_description>

<severity>moderate</severity>
<identifiers>GHSA-chgr-c6px-7xpp</identifiers>
<package>pyo3</package>
<ecosystem>rust</ecosystem>
<vulnerable_versions>= 0.28.3</vulnerable_versions>
<patched_version>0.29.0</patched_version>
<manifest_path>bindings/python/Cargo.lock</manifest_path>

<references>
<url>https://github.com/PyO3/pyo3/pull/6096</url>
<url>https://github.com/PyO3/pyo3/releases/tag/v0.29.0</url>
<url>https://rustsec.org/advisories/RUSTSEC-2026-0177.html</url>
<url>https://github.com/advisories/GHSA-chgr-c6px-7xpp</url>
</references>

<agent_instructions>Look at how other projects have implemented the fix and use that to fix this issue.</agent_instructions>

<task_instructions>Resolve this alert by updating the affected package to a non-vulnerable version. Prefer the lowest non-vulnerable version (see the patched_version field above) over the latest to minimize breaking changes. Include a Reachability Assessment section in the PR description. Review the alert_description field to understand which APIs, features, or configurations are affected, then search the codebase for usage of those specific items. If the vulnerable code path is reachable, explain how (which files, APIs, or call sites use the affected functionality) and note that the codebase is actively exposed to this vulnerability. If the vulnerable code path is not reachable, explain why (e.g. the affected API is never called, the vulnerable configuration is not used) and note that the update is primarily to satisfy vulnerability scanners rather than to address an active risk. If the advisory is too vague to determine reachability (e.g. 'improper input validation' with no specific API named), state that reachability could not be determined and explain why. Include a confidence level in the reachability assessment (e.g. high confidence if the advisory names a specific API and you confirmed it is or is not called, low confidence if the usage is indirect and hard to trace). If no patched version is available, check the alert_description field for a Workarounds section — the advisory may describe configuration changes or usage patterns that mitigate the

vulnerability without a version update. If a workaround is available, apply it and leave a code comment referencing the advisory identifier explaining it is a temporary mitigation. If neither a patch nor a workaround is available, explain in the PR description why the alert cannot be resolved automatically so a human reviewer can take over. Inspect the repository to determine which package manager is used (e.g. lock files, config files, build scripts) and use that tooling to perform the update — do not edit lock files directly. If the version constraint in the manifest (e.g. package.json, Gemfile, pyproject.toml) caps the version below the fix, update the constraint first. For transitive dependencies, determine whether it is simpler to update the direct dependency that pulls in the vulnerable package or to update the transitive dependency directly, and choose the least disruptive approach. If upgrading to fix the vulnerability forces a major version bump or known breaking changes, review the changelog or release notes, then audit the codebase for usage of affected APIs and fix any breaking changes that are found. If the package manager fails to resolve dependencies (e.g. peer dependency conflicts, incompatible engine constraints), document the error in the PR description rather than attempting increasingly complex workarounds. After updating, check the lock file to confirm the package no longer resolves to a version in the vulnerable range. Keep changes minimal and tightly scoped. Ensure tests, build, type checking, and linting all pass after your changes. If there are any test, lint, or typechecking failures, investigate whether they are caused by the update and fix them if so — do not leave broken tests in the PR. If they were already present before the update, note them in the PR description so a human reviewer can assess whether they are related.</task_instructions>

</problem_statement>

Implement the necessary changes to the repository so that the requirements specified in the <problem_statement> are met.

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.